



SMARTDOC AI — HƯỚNG DẪN THUYẾT TRÌNH 10 CÂU MỞ RỘNG

Môn: Open Source Software Development · Spring 2026

Mục tiêu: Phân tích chi tiết luồng hoạt động, hàm nào làm gì, biến nào trả về



TỔNG QUAN KIẾN TRÚC FILE

app.py< UI Streamlit, điều phối toàn bộ luồng

rag_engine.py< Logic RAG, Self-RAG, Rerank, Graph RAG, Hybrid Search

rag_engine_graph_optimized.py< Build Knowledge Graph song song (tối ưu)

database.py< SQLite lưu lịch sử hội thoại

? CÂU HỎI 1 — Thêm hỗ trợ file DOCX

Luồng hoạt động

User upload .docx

↓

app.py: kiểm tra ext = ".docx"

↓

rag_engine.process_docx(tmp_path, embedder, chunk_size, chunk_overlap)

↓

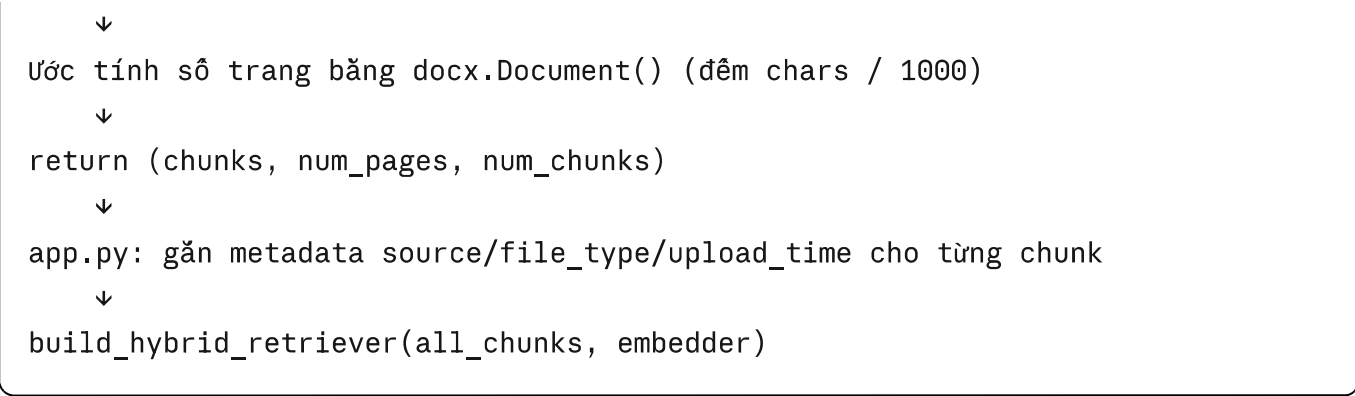
Docx2txtLoader.load() → list[Document]

↓

_clean_text() cho từng doc

↓

RecursiveCharacterTextSplitter.split_documents()



Hàm chính

`process_docx(file_path, embedder, chunk_size, chunk_overlap)` — `rag_engine.py`

Bước	Thư viện	Mô tả
Load	<code>Docx2txtLoader</code>	Trích toàn bộ text từ DOCX
Clean	<code>_clean_text()</code>	Xóa ký tự control, chuẩn hóa newline, xóa số trang
Split	<code>RecursiveCharacterTextSplitter</code>	Chia theo <code>\n\n → \n → . → ! → ?</code>
Filter	list comprehension	Bỏ chunk < 50 ký tự
Count pages	<code>docx.Document()</code>	Đếm tổng chars / 1000 = ước tính trang

Trả về: `((chunks: list[Document], num_pages: int, num_chunks: int))`

Điểm khác so với PDF

- PDF dùng `PDFPlumberLoader` (giữ thông tin trang, table layout)

- DOCX dùng `Docx2txtLoader` + `docx.Document()` để đếm trang ước tính (không có page metadata thật)
-

? CÂU HỎI 2 — Lưu trữ lịch sử hội thoại

Luồng hoạt động

```
App khởi động
↓
database.init_db() ← tạo bảng conversations nếu chưa có + auto migration cột
sources
↓
Mỗi lần trả lời xong:
↓
database.save_message(session_id, pdf_name, question, answer, sources)
↓
SQLite INSERT → bảng conversations
↓
Sidebar: load_all_sessions() → hiển thị danh sách session
↓
Khi click session: load_history(session_id) → hiển thị Q&A
```

Hàm chính — `database.py`

`init_db()`

- Tạo bảng `conversations(id, session, pdf_name, question, answer, sources, timestamp)`
- **Auto-migration:** kiểm tra `PRAGMA table_info` → `ALTER TABLE ADD COLUMN sources` nếu DB cũ

- `sources` là list dict → `json.dumps()` trước khi lưu
- `timestamp` = `datetime.now().strftime("%Y-%m-%d %H:%M:%S")`

`load_history(session_id) → list[tuple]`

- `SELECT question, answer, sources, timestamp WHERE session=? ORDER BY id`
- Trả về: `list of (question, answer, sources_json_str, timestamp)`

`load_all_sessions() → list[tuple]`

- `SELECT session, MIN(question), COUNT(*), MIN(timestamp) GROUP BY session ORDER BY started DESC LIMIT 20`
- Hiển thị sidebar: session_id + câu hỏi đầu tiên + số câu + ngày bắt đầu

Session State trong app.py

```
python

st.session_state.session_id = str(uuid.uuid4())[:8] # ID ngắn 8 ký tự
st.session_state.chat_history = [] # list dict {question, answer, sources, tim
```

? CÂU HỎI 3 — Nút xóa lịch sử + xóa tài liệu

Luồng hoạt động

```
Nút "Xóa tất cả lịch sử"
↓
@st.dialog("Xóa tất cả lịch sử?") ← popup xác nhận
```

↓
Xác nhận → `clear_all_history()`
↓
`DELETE FROM conversations` (xóa toàn bộ)
↓
`st.session_state.chat_history = []`
↓
`_mark_sessions_dirty()` → sidebar reload

Nút "🗑️ Xóa tài liệu"
↓
`@st.dialog("Xóa tài liệu đang tải?")` ← popup xác nhận
↓
Xác nhận → reset session state:
 `retriever = None`
 `graph = None`
 `chunks_store = None`
 `pdf_info = None`
 `documents = []`
 `uploader_key += 1` ← reset widget uploader

Hàm chính — `database.py`

`clear_all_history()` → `DELETE FROM conversations`

`delete_session(session_id)` → `DELETE FROM conversations WHERE session=?`

Cơ chế Confirmation Dialog — `app.py`

- Dùng `@st.dialog()` decorator — Streamlit native modal
- Không cần JS/HTML thêm
- `st.rerun()` sau khi xóa để refresh UI

? CÂU HỎI 4 — Cải thiện Chunk Strategy

Luồng hoạt động

```
Sidebar: selectbox chunk_size [500, 1000, 1500, 2000]
         selectbox chunk_overlap [50, 100, 200, 300]
         ↓
st.session_state.chunk_size / chunk_overlap cập nhật
         ↓
Khi upload file mới → process_pdf() / process_docx() dùng giá trị mới
         ↓
Nếu params thay đổi sau khi đã upload → hiển thị warning ⚠
```

Hàm chính

`RecursiveCharacterTextSplitter` trong `process_pdf()` và `process_docx()`

```
python

splitter = RecursiveCharacterTextSplitter(
    chunk_size=chunk_size,      # 500 / 1000 / 1500 / 2000
    chunk_overlap=chunk_overlap, # 50 / 100 / 200 / 300
    separators=["\n\n", "\n", ".", "!", "?", ",", " ", " ", ""],
)
```

Giải thích separators: Cắt theo đoạn văn trước (`\n\n`), rồi dòng (`\n`), rồi câu (`.`, `!`, `?`), rồi từ (`,`,), cuối cùng ký tự. Đảm bảo không cắt giữa câu nếu có thể.

Tham số	Giá trị nhỏ	Giá trị lớn
chunk_size	Retrieval chính xác hơn, context ít hơn	Context nhiều hơn, retrieval rộng hơn
chunk_overlap	Nguy cơ mất ngữ cảnh ranh giới	Tốn RAM hơn, trùng lặp nhiều hơn

? **CÂU HỎI 5 — Citation / Source Tracking**

Luồng hoạt động

```
ask_question_stream_with_sources() stream token về app.py
↓
Gặp token bắt đầu bằng "@@SOURCES@"
↓
sources_data = json.loads(token[len("@@SOURCES@"):])
↓
_render_sources(sources_data, question, answer)
↓
Với mỗi source: _highlight_text(content, question, answer)
↓
N-gram Jaccard scoring → highlight câu nguồn bằng <mark> vàng
```

Hàm chính

`ask_question_stream_with_sources()` — build sources list:

```
python
```

```
sources = [
    {
        "index": i + 1,
        "page": doc.metadata.get("page") + 1, # 0-indexed → 1-indexed
        "source": os.path.basename(doc.metadata.get("source")),
        "content": doc.page_content,
    }
    for i, doc in enumerate(source_docs)
]
yield "@@SOURCES@" + json.dumps(sources)
```

`_highlight_text(content, question, answer)` — thuật toán N-gram Source Tracing:

1. **Tách câu** trong chunk (≥ 40 ký tự, dùng regex dấu câu tiếng Việt)
2. **Tách câu** trong answer (≥ 20 ký tự)
3. **Bigram + Trigram Jaccard**: với mỗi câu chunk, tính `max_jaccard` so với mọi câu trong answer
 - Bigram weight: 0.4; Trigram weight: 0.6 (trigram đặc trưng hơn)
4. **Dynamic threshold**: chỉ highlight nếu score $\geq \max(0.05, \text{best_score} \times 0.5)$
5. **Tối đa 2 câu highlight / chunk** — tránh over-highlight
6. **Span-based rendering**: tìm vị trí ký tự trong content gốc → bọc `<mark>` HTML

`_render_sources()`: Hiển thị trong `st.expander()` với border xanh, font nhỏ

Luồng hoạt động

Mỗi câu hỏi mới:

↓

`ask_question_stream_with_sources(question, retriever, chat_history=[...])`

↓

`_format_chat_history(chat_history, max_turns=4)`

↓

Format thành text: "[Người dùng]: ...\n[Trợ lý]: ..."

↓

Đưa vào prompt tại vị trí {chat_history}

↓

LLM thấy ngữ cảnh hội thoại → xử lý follow-up question

Hàm chính

`_format_chat_history(chat_history, max_turns=4)`

- Lấy `chat_history[-4:]` (4 turn gần nhất)
- Truncate answer > 350 ký tự (cắt theo từ cuối, thêm `...`)
- Skip compare-mode answers (chứa `**Classic RAG:**`)
- Format: `[Người dùng]: {q}\n[Trợ lý]: {a}`
- **Trả về:** string đưa thẳng vào prompt

Prompt template — `_build_prompt()`:

```
[HỢI THOẠI TRƯỚC]
{chat_history}
```

```
[NGŨ CẢNH]
{context}
```

```
[CÂU HỎI]
{question}
```

Hướng dẫn trong prompt: "Với câu hỏi follow-up → kết hợp [HỢI THOẠI TRƯỚC] và [NGŨ CẢNH] để giữ ngữ mạch liên tục"

? CÂU HỎI 7 — Hybrid Search (FAISS + BM25)

Luồng hoạt động

```
build_hybrid_retriever(all_chunks, embedder)
    ↓
FAISS.from_documents(chunks, embedder)
    → faiss_retriever (MMR, fetch_k = k×3)
    ↓
BM25Retriever.from_documents(chunks)
    → bm25_retriever (k = top_k)
    ↓
EnsembleRetriever(
    retrievers=[bm25_retriever, faiss_retriever],
    weights=[0.3, 0.7]
)
    ↓
Khi query: bm25 lấy k docs + faiss lấy k docs
```

- Reciprocal Rank Fusion merge kết quả
- Trả về list đã re-ranked

Hàm chính

`build_hybrid_retriever(chunks, embedder)` — `rag_engine.py`

Thành phần	Loại	Trọng số	Đặc điểm
FAISS (MMR)	Semantic	0.7	Dense vector, tìm theo ý nghĩa
BM25	Keyword	0.3	Sparse, tốt với từ khóa chính xác

Tại sao FAISS = 0.7? Câu hỏi tiếng Việt thường diễn đạt đa dạng → semantic search quan trọng hơn exact keyword match.

MMR (Maximal Marginal Relevance):

```
python

faiss_retriever = vector_store.as_retriever(
    search_type="mmr",
    search_kwargs={"k": top_k, "fetch_k": top_k * 3},
)
```

Lấy `fetch_k` docs → chọn `k` docs đa dạng nhất (tránh trùng lặp nội dung).

Reciprocal Rank Fusion (do LangChain EnsembleRetriever tự làm): Mỗi doc được score = `sum(weight_i / (rank_i + 60))` → merge list.

? CÂU HỎI 8 — Multi-document RAG + Metadata Filtering

Luồng hoạt động

```
Upload nhiều file (accept_multiple_files=True khi không compare mode)
↓
Lặp qua từng file:
    process_pdf() / process_docx()
    Gắn metadata: source=file.name, file_type, upload_time
    all_chunks.extend(chunks)
↓
build_hybrid_retriever(all_chunks, embedder) ← tất cả chunks vào 1 retriever
↓
pdf_info["names"] = [f1.name, f2.name, ...]
↓
Sidebar: selectbox "Filter theo tài liệu" = ["All", f1, f2, ...]
↓
ask_question_stream_with_sources():
    if selected_file != "All":
        retrieved_docs = [d for d in docs if d.metadata["source"] ==
selected_file]
```

Metadata được lưu trong chunk

```
python

chunk.metadata = {
    "page": int,          # từ PDFPlumber (0-indexed)
    "source": "file.pdf", # gán bởi app.py
    "file_type": "pdf",   # "pdf" | "docx"
    "upload_time": str,   # datetime string
}
```

- `_render_sources()` show tên file từ `src["source"]`
 - `_build_context()` build header `[Đoạn N – Trang X | file.pdf]`
-

? CÂU HỎI 9 — Re-ranking với Cross-Encoder

Luồng hoạt động

```
Sidebar: checkbox "Bật Re-ranking (Cross-Encoder)"
    ↓
CONFIG["use_rerank"] = True
    ↓
ask_question_stream_with_sources():
    retrieved_docs = retriever.invoke(question)[:20] ← lấy nhiều ứng viên
    ↓
if CONFIG["use_rerank"]:
    source_docs = rerank_documents(question, retrieved_docs, top_k=k)
else:
    source_docs = retrieved_docs[:k]
```

Hàm chính

`get_cross_encoder()` — singleton

```
python

_cross_encoder = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")
```

Model nhỏ (22M params), fast, tối ưu cho MS MARCO passage ranking.

`rerank_documents(question, docs, top_k=5)`

```
python

pairs = [(question, doc.page_content) for doc in docs]
scores = cross_encoder.predict(pairs)  # mảng float score
for doc, score in zip(docs, scores):
    doc.metadata["rerank_score"] = float(score)
docs = sorted(docs, key=lambda x: x.metadata["rerank_score"], reverse=True)
return docs[:top_k]
```

Bi-encoder vs Cross-Encoder

	Bi-encoder (FAISS)	Cross-Encoder (Rerank)
Input	Query riêng, Doc riêng	(Query, Doc) cùng lúc
Output	Vector similarity	Relevance score tuyệt đối
Tốc độ	Nhanh (O(1) sau index)	Chậm (O(n) per query)
Độ chính xác	Trung bình	Cao hơn
Dùng khi	Candidate retrieval	Final ranking

Pipeline 2 tầng: Bi-encoder lấy 20 ứng viên → Cross-encoder chọn top-k chính xác nhất.

? CÂU HỎI 10 — Self-RAG

Luồng hoạt động

```
Sidebar: chọn "Self-RAG"
↓
```

```

CONFIG["use_self_rag"] = True
↓
ask_question_stream_with_sources() → gọi self_rag_pipeline()
↓
Vòng lặp (max 2 lần):
    Vòng 1: search_query = question gốc
    Vòng 2+: search_query = rewrite_query(question + missing_hint)
↓
    docs = retriever.invoke(search_query)[:10]
    docs = rerank hoặc top-k
    context = _build_context(docs)
    answer = llm.invoke(prompt)
↓
    eval = evaluate_answer(question, answer, context)
    score = eval["score"]
↓
    if score > best_score: cập nhật best
    if score > 0.8: break sớm
↓
return best_answer, best_score, best_context, best_docs

```

Ba hàm cốt lõi

1. `rewrite_query(question, chat_history)` → `str`

Dùng LLM viết lại câu hỏi cho phù hợp hơn với RAG retrieval.

5 chiến lược trong prompt:

1. Mở rộng từ viết tắt
2. Thêm loại thực thể ngầm hiểu
3. Tách câu phức hợp thành câu con có thể tìm kiếm nhất
4. Thay thế đại từ (nó, họ, điều này) bằng tên cụ thể từ lịch sử

5. Giữ nguyên ngôn ngữ gốc

Trả về: string câu hỏi đã rewrite (chỉ lấy dòng đầu của LLM response)

2. `evaluate_answer(question, answer, context)` → `dict`

Dùng LLM đánh giá chất lượng câu trả lời theo rubric:

Score	Ý nghĩa
0.9-1.0	Đầy đủ, chính xác, có ngữ cảnh hỗ trợ hoàn toàn
0.7-0.8	Phần lớn đúng, thiếu sót nhỏ
0.5-0.6	Trả lời một phần, có thông tin không xác nhận được
0.3-0.4	Mơ hồ, phần lớn không liên quan
0.0-0.2	Sai hoặc bịa đặt

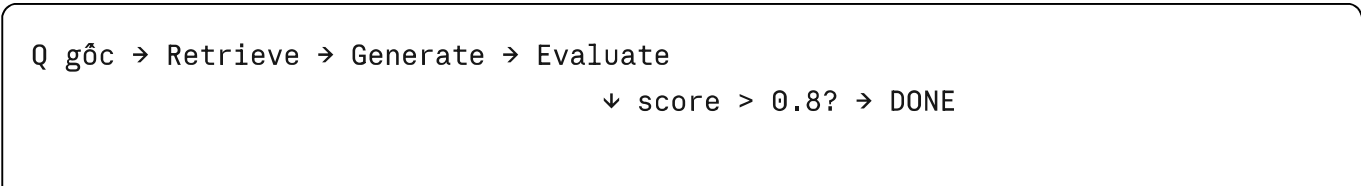
Trả về: `{"score": float, "missing": str, "reason": str}`

`missing` = thông tin còn thiếu → truyền vào `rewrite_query()` vòng sau để search thông minh hơn.

3. `self_rag_pipeline(question, retriever, chat_history)` → `(answer, score, context, docs)`

Kết hợp hai hàm trên trong vòng lặp, trả về câu trả lời tốt nhất.

Sơ đồ Self-RAG



↓ score ≤ 0.8 + còn vòng lặp?
Rewrite Query (dùng "missing") → Retrieve lại → ...

NEW

ĐẶC BIỆT — So sánh Classic RAG vs Graph RAG

Tổng quan hai phương pháp

	Classic RAG (Hybrid)	Graph RAG
Retrieval	Vector similarity + BM25	Entity graph traversal
Index	FAISS dense vector	NetworkX knowledge graph
Tìm kiếm	"Đoạn nào gần nghĩa nhất?"	"Entity nào liên quan? → láng giềng?"
Mạnh ở	Câu hỏi cụ thể, keyword rõ	Câu hỏi về quan hệ, suy luận đa bước
Build time	Nhanh (~5s)	Chậm (90-120s cold, <3s từ cache)

LUỒNG BUILD KNOWLEDGE GRAPH

Bật toggle "So sánh RAG vs Graph RAG" hoặc upload file khi compare mode
↓
app.py → streamlit_build_graph_with_progress(all_chunks)
↓
rag_engine_graph_optimized.build_graph_rag_fast(chunks, llm, embedder)

Bước 1: Pre-filter chunks

Bước 2: Cache check

```
python

chunks_hash = _compute_chunks_hash(filtered) # SHA-256 16 ký tự
cached = _load_cached_graph(chunks_hash)      # load từ .graph_cache/graph_{ha
```

Nếu cache HIT → return ngay, bỏ qua toàn bộ phần build.

Bước 3: Parallel LLM Extraction (ThreadPoolExecutor)

```
python

with ThreadPoolExecutor(max_workers=4) as executor:
    futures = {executor.submit(_process_chunk, (idx, chunk)): idx ...}
    for future in as_completed(futures):
        chunk_data, rels = future.result()
        # Build graph (thread-safe với GIL)
```

`_extract_entities_fast(text, llm, max_rels=6)`:

- Dùng prompt tối giản (giảm ~30-40% latency so với prompt đầy đủ)
- Cắt text xuống 600 ký tự trước khi gửi LLM
- Parse JSON array `[{"e1":..., "rel":..., "e2":...}]`
- Filter entity generic (`_GENERIC_ENTITIES`) và entity < 3 ký tự
- **Fallback nếu LLM fail:** regex tìm ACRONYM + TitleCase

Bước 4: Build NetworkX Graph

```
python

G = nx.Graph()
# Với mỗi relation (e1, rel, e2):
G.add_node(e1, chunks=[chunk_data], label=e1)
G.add_node(e2, chunks=[chunk_data], label=e2)
G.add_edge(e1, e2, weight=1, rels=[relationship])
# Nếu edge đã tồn tại: weight += 1
```

Bước 5: Node Embedding (batch)

```
python

node_embeddings = embedder.embed_documents(node_list) # 1 lần duy nhất
for node, emb in zip(node_list, node_embeddings):
    G.nodes[node]["embedding"] = emb
```

Bước 6: Save cache

```
python

pickle.dump(G, f) # lưu vào .graph_cache/graph_{hash}.pkl
```

LƯỚI GRAPH RAG RETRIEVAL

`_graph_retrieve(question, graph, top_k=6)` — 3 tầng:

Tầng 1: Semantic Node Matching

```
python
```

```
q_emb = embedder.embed_query(question)
for node, node_emb in nodes_with_emb:
    sim = cosine_similarity(q_emb, node_emb)
    if sim >= 0.25:
        seed_nodes[node] = sim
seed_nodes = top-5 by score
```

→ Tìm node nào "gần nghĩa" nhất với câu hỏi.

Tầng 2: Graph Traversal (Neighbor Expansion)

```
python

for node, sim in seed_nodes.items():
    nbrs = sorted(graph.neighbors(node), by edge weight, top 3)
    for nb in nbrs:
        nb_score = sim * 0.6 * (edge_weight / max_weight)
        neighbor_nodes[nb] = nb_score
```

→ Mở rộng sang các node láng giềng, score giảm dần theo độ xa.

Tầng 3: Chunk Deduplication + Ranking

```
python

# Thu thập chunks từ seed nodes (score cao) + neighbor nodes (score thấp hơn)
chunk_scores[chunk_idx] = (best_score, chunk_data)
ranked = sorted(chunk_scores.values(), by score, top top_k)
```

→ Trả về list `chunk_data` dict (không phải LangChain Document).

Fallback chain:

1. Nếu không có node nào có embedding → keyword matching theo tên node

2. Nếu không có seed node → brute-force keyword scoring trên tất cả chunks trong graph

LƯỚI SO SÁNH SONG SONG — ask_compare_rag()

```
app.py: user gõ câu hỏi, compare_mode=True
      ↓
ask_compare_rag(question, retriever, graph, chat_history)
      ↓
[Generator yield tokens]
      ↓
"@@CLASSIC_START@"           → app.py biết bắt đầu classic
... token classic ...
"@@CLASSIC_SOURCES@[...]"    → app.py hiển thị sources classic
      ↓
"@@GRAPH_START@"             → app.py biết bắt đầu graph
... token graph ...
"@@GRAPH_SOURCES@[...]"      → app.py hiển thị sources graph
      ↓
"@@COMPARE_DONE@"            → app.py kết thúc loop
```

Xử lý ở app.py:

```
python
```

```
for token in ask_compare_rag(...):
    if token == "@@CLASSIC_START@@": phase = "classic"
    elif token.startswith("@@CLASSIC_SOURCES@@"): hiển thị sources classic
    elif token == "@@GRAPH_START@@": phase = "graph"
    elif token.startswith("@@GRAPH_SOURCES@@"): hiển thị sources graph
    elif token == "@@COMPARE_DONE@@": break
    else:
        if phase == "classic": classic_answer += token; classic_placeholder.mar
        elif phase == "graph": graph_answer += token; graph_placeholder.markdow
```

2 cột song song:

- Cột trái: Classic RAG streaming
- Cột phải: Graph RAG (invoke sau khi classic xong)

 BẢNG TÓM TẮT — HÀM & BIÊN QUAN TRỌNG

Hàm	File	Input	Out
<code>process_pdf()</code>	rag_engine	file_path, embedder, chunk_size, chunk_overlap	(chu nun
<code>process_docx()</code>	rag_engine	file_path, embedder, chunk_size, chunk_overlap	(chu nun
<code>init_db()</code>	database	—	—

Hàm	File	Input	Out
<code>save_message()</code>	database	session_id, pdf_name, q, a, sources	—
<code>load_history()</code>	database	session_id	list[
<code>clear_all_history()</code>	database	—	—
<code>build_hybrid_retriever()</code>	rag_engine	chunks, embedder	Ens
<code>ask_question_stream_with_sources()</code>	rag_engine	question, retriever, chat_history	Gen
<code>rerank_documents()</code>	rag_engine	question, docs, top_k	list[
<code>self_rag_pipeline()</code>	rag_engine	question, retriever, chat_history	(ans con
<code>rewrite_query()</code>	rag_engine	question, chat_history	str
<code>evaluate_answer()</code>	rag_engine	question, answer, context	{"sc "rea
<code>build_graph_rag_fast()</code>	rag_engine_graph_optimized	chunks, llm, embedder	nx.C

Hàm	File	Input	Out
<code>_graph_retrieve()</code>	rag_engine	question, graph, top_k	list[dict]
<code>ask_compare_rag()</code>	rag_engine	question, retriever, graph, chat_history	Gen
<code>_highlight_text()</code>	app	content, question, answer	HTM
<code>_format_chat_history()</code>	rag_engine	chat_history, max_turns=4	str

 GỢI Ý KHI THUYẾT TRÌNH

Thứ tự trình bày đề xuất

- 1. **Demo live** (nếu được): Upload PDF → hỏi → show sources → highlight
- 2. **Giải thích Q1-Q3** (dễ, cơ bản)
- 3. **Q4** — show chunk params thay đổi ảnh hưởng gì
- 4. **Q5** — giải thích thuật toán N-gram highlight (điểm sáng kỹ thuật)
- 5. **Q6** — Conversational RAG (format_chat_history)
- 6. **Q7** — Hybrid Search sơ đồ BM25+FAISS
- 7. **Q9** — Rerank 2 tầng, bảng so sánh bi vs cross encoder
- 8. **Q10** — Self-RAG vòng lặp, rubric evaluate

9. **Compare mode** — Graph RAG build + 3-tầng retrieval + stream protocol (điểm nổi bật nhất)

Câu hỏi thầy có thể hỏi

"Tại sao dùng Jaccard bigram thay vì BM25 để highlight?"

→ BM25 là bag-of-words → "Hồ Chí Minh" score cao ở mọi câu. Bigram phân biệt cụm từ đặc trưng hơn.

"Graph RAG lấy nhiều context hơn Classic RAG không?"

→ Lấy $\text{top_k} + 3$ chunks (thêm 3 so với Classic). Lý do: graph traversal đã pre-filter theo relevance → cần ít context thêm để tổng hợp quan hệ.

"Tại sao cache graph bằng SHA-256 hash của chunks?"

→ Nếu content file không đổi → hash không đổi → skip rebuild 90-120s → chi mất < 3s.

"Self-RAG max 2 vòng có đủ không?"

→ Đủ cho local CPU (tránh quá chậm). Mỗi vòng gọi LLM 3 lần (rewrite + generate + evaluate). 2 vòng = tối đa 6 LLM calls.

"Tại sao EnsembleRetriever weights [0.3, 0.7]?"

→ Tiếng Việt diễn đạt đa dạng → semantic (FAISS) quan trọng hơn exact keyword (BM25).

Tài liệu này được tạo dựa trên phân tích trực tiếp source code: `app.py`, `rag_engine.py`, `rag_engine_graph_optimized.py`, `database.py`